A Source-Agnostic AI-Powered Smart City Observatory: Self-Generating, Self-Healing, and Self-Deploying Data Wrappers in Production

Anonymous Author(s)

***Abstract*—A smart city observatory is only as useful as the breadth of sources it can ingest, yet most indicator platforms hard-wire a handful of feeds and pay an engineer to maintain a wrapper for each. We present the Observatory, a production smart city observatory whose ingestion layer is source-agnostic: any REST API, spreadsheet, or CSV export becomes a normalized indicator time series with no source-specific code written by a human. Each source is handled by a data wrapper that we treat as an autonomic artifact with three properties. It is *self-generating*: a large language model fills a fixed extraction template from source metadata and a data sample, probing live endpoints before committing to extraction logic. It is *self-healing*: a monitor re-invokes the model with the crash traceback and repairs the wrapper in place. And it is *self-deploying*: wrappers are launched, supervised, checkpointed, and relaunched as isolated processes with no operator in the loop. We frame the three properties as a single autonomic manager over the generated artifact and report a 149-day production deployment: 396 wrappers across API, XLSX, and CSV sources ingested 164,322 data points; 43 wrappers (10.9%) crashed and entered the repair loop, after which 42 of 43 crashed no further at a median of 33.4 s per repair; and the whole platform runs in roughly 1.5 GiB of memory. The boundary of the approach is sharp: the loops absorb heterogeneity and code defects, while quota, credential, and configuration failures stay outside their reach.**

***Index Terms*—smart city observatory, source-agnostic data ingestion, autonomic computing, self-healing systems, LLM code generation, ETL automation, smart tourism**

I. Introduction

Smart city and destination observatories promise a single pane of glass over heterogeneous urban indicators: air and water quality, mobility, energy, overnight stays, cultural participation. The promise is bounded in practice by the ingestion layer. Each indicator is published by a different statistical office, municipality, or sector association, as a REST API, an Excel workbook, or an ad hoc CSV export, with its own schema, date format, language, and failure modes. The classical answer is one extraction wrapper per source, but writing and maintaining those wrappers is the cost that has limited mediator-based data integration since the 1990s [1]–[3]. A platform that can only afford a fixed set of wrappers is, in effect, not an observatory of its city but of the few sources someone had time to integrate.

We call an ingestion layer *source-agnostic* when adding a new source costs no source-specific engineering: a domain user registers the source through a web form, and the platform produces, runs, and maintains the wrapper on its own. This paper describes the Observatory (anonymized for review), a production smart city observatory for a European region in which source-agnostic ingestion is realized by treating each wrapper as an *autonomic* artifact in the sense of Kephart and Chess [4]: a self-managing component with a closed control loop over its own lifecycle.

Concretely, every wrapper has three self-* properties. *Self-generating*: when a source is registered, a large language model (LLM) fills a fixed extraction template from the indicator metadata and a sample of the data, and for live APIs it probes the endpoint through a controlled fetch tool before writing code (Section IV-A). *Self-healing*: generated code fails in production, and a monitor re-invokes the model with the failing source and its captured stack trace, redeploying the repaired wrapper automatically (Section IV-C). *Self-deploying*: wrappers run as isolated operating-system processes that the platform spawns, supervises, checkpoints, and relaunches without an operator, including resume-from-watermark after restarts and automatic image updates in production (Section IV-B).

This paper makes three contributions. First, we give a source-agnostic observatory architecture and the contract that makes it work: any source is reduced to a stream of normalized $(\text{timestamp}, \text{value})$ points by a generated wrapper that the platform owns end to end (Section III). Second, we formalize the wrapper as a single autonomic manager and separate its three self-* loops, relating each to the relevant line of prior work in autonomic computing, LLM code generation and repair, and wrapper maintenance (Section IV). Third, we report measurements from a real 149-day deployment rather than a benchmark, including a natural pre-repair baseline and a failure taxonomy that marks exactly where the autonomic loops stop helping (Section V). Anonymized source mirrors will be linked here for review.

II. Background and Related Work

A. Smart city and tourism observatories

Smart tourism research calls for destination-level data infrastructures [5], [6], indicator schemes such as ETIS define what such platforms should measure [7], and implementation

studies report uneven adoption [8] and warehouse- or survey-based dashboards [9]. Studies of smart destination programs repeatedly identify data availability for sustained indicator monitoring as the bottleneck [10], [11]. Reported platforms rarely document the ingestion layer; we argue that source-agnostic ingestion is the property that decides how many indicators an observatory can actually sustain, and we document one in production.

B. Autonomic computing and self-* systems

The autonomic computing vision frames self-managing components with closed control loops as the answer to operational complexity [4], with self-configuration, self-healing, and self-optimization as canonical properties. the Observatory instantiates this vision for data wrappers: generation is self-configuration of a new artifact, repair is self-healing, and process supervision is self-deployment. The novelty is the actuator: an LLM that writes and rewrites the artifact's code, rather than tuning parameters of a fixed one.

C. LLMs for code generation, repair, and extraction

LLMs generate working programs from natural language [12], and program repair is increasingly LLM-based [13]. Feedback loops improve generated code: Self-Refine [14], Reflexion [15], and Self-Debugging [16], though self-repair gains are not uniform [17]. On the data side, Evaporate showed that generated extraction code amortizes inference cost across documents [18], AutoScraper generates reusable scrapers [19], and prompted [20] and instruction-tuned [21] models perform classical data preprocessing. We differ from generation-time self-repair in the trigger and the unit: repair fires on a production crash, possibly weeks after generation, and rewrites the whole wrapper file rather than patching a line.

D. Wrapper induction, maintenance, and ETL automation

Wrapper induction learned extraction rules from labeled examples [2]; unsupervised systems inferred them from page regularities [22]; surveys cover the landscape [23], [24]. The maintenance problem, detecting breakage and re-inducing the wrapper, was recognized early [3], [25]. ETL has been modeled rather than synthesized [26], [27], with automation of schema matching [28], curation [29], and wrangling [30]. the Observatory replaces both induction and re-induction with LLM code generation, extending coverage beyond template pages to arbitrary APIs and spreadsheets, and automates the narrow but recurrent ETL slice indicator platforms need: single-series extraction into one normalized time-series schema.

III. A Source-Agnostic Observatory

The Observatory is a microservice platform: containerized user, knowledge-base, data-collector, resource, and indicator services (Python, FastAPI) behind an nginx gateway, plus a React dashboard. Two RabbitMQ brokers separate the control plane from the data plane; the resource and indicator services each own a MongoDB store. Figure 1 shows the pipeline.

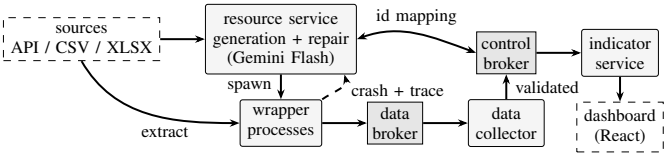


Fig. 1. The Observatory ingestion pipeline. The resource service generates, deploys, and repairs wrappers; extracted points flow through two brokers to the indicator store and the dashboard.

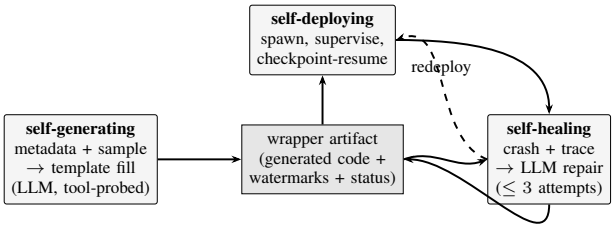


Fig. 2. The autonomic wrapper manager. Three self-* loops act on one shared artifact; the actuator is an LLM that writes and rewrites code, plus a process supervisor that deploys it.

The property that makes ingestion source-agnostic is a narrow contract. A wrapper, whatever its source, publishes batches of points $\{(x, y)\}$ with x an ISO 8601 timestamp and y a float; everything upstream of that contract is generated, and everything downstream is fixed, shared infrastructure. A domain user registers a source in a web wizard, either uploading a CSV/XLSX file or configuring an API endpoint with its authentication, and supplies indicator metadata (name, domain, unit, periodicity, description). The resource service then owns the wrapper for its whole lifecycle. The data collector validates each message against the contract, caches the latest per wrapper, and forwards valid points to the control broker, where the resource service maps wrapper identifiers to persistent resources and the indicator service stores, merges, and serves the series.

IV. Self-* Wrappers

We model each wrapper as an autonomic manager whose managed element is the generated artifact itself: its code, its execution watermarks, and its status. The manager runs three loops over shared knowledge (Figure 2). The plan-and-execute step of each loop is an LLM call or a process action; the monitored knowledge is the wrapper's runtime behavior, not a synthetic test suite. We take the loops in lifecycle order.

A. Self-generating

Generation is constrained, not free-form. The model fills a fixed wrapper class with a small number of variable methods (extraction, and for APIs the date-range fetch and polling interval); transport, batching, resumption, and the historical-backfill driver are fixed library code the model is told not to touch. The prompt combines the indicator metadata, the source configuration, a data sample (first lines of a CSV, rows per sheet of a workbook, or a captured API response),

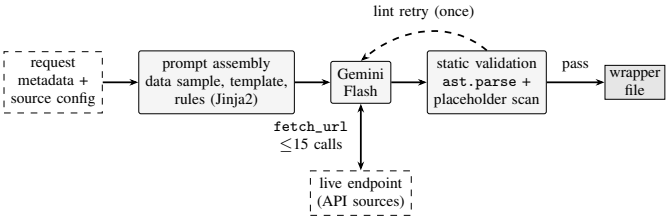


Fig. 3. Self-generation. The prompt combines metadata, a data sample, the fixed template, and hard rules; for API sources the model probes the live endpoint before writing code; output must pass static validation, with one retry, before deployment.

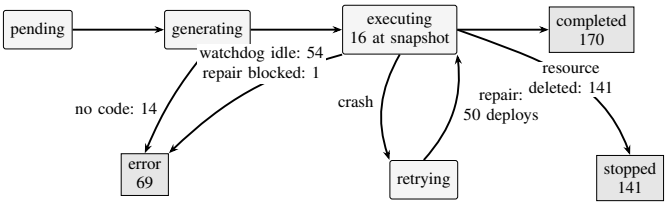


Fig. 4. Wrapper status lifecycle with production counts ($n = 396$). The crash–repair cycle covered 43 wrappers and 50 repair deployments; most `error` states are watchdog terminations of idle API pollers, not code defects.

the required output schema, the template, and a list of hard rules that encode failure modes seen in development (inclusive date filtering, consistent timezones, no synthetic data, explicit parsing of non-English dates). For API sources the generation call exposes a controlled `fetch_url` tool, so the model discovers response nesting, field names, and which date-filter parameters the endpoint honors from the live endpoint rather than guessing. Before deployment the code is parsed and scanned for unresolved placeholders, with one lint retry; there is no pre-flight test, runtime faults are delegated to self-healing. Figure 3 shows the flow.

B. Self-deploying

A generated wrapper reaches production with no human action. The resource service starts each wrapper as a separate operating-system process, with concurrency bounded by a semaphore and a per-process execution watchdog. File wrappers run once and exit; API wrappers first backfill history in adaptive batches, then poll at the interval implied by the indicator periodicity, with no external scheduler. The service persists two watermarks per wrapper, the oldest and newest timestamps ever sent, so that after a restart a wrapper resumes from its watermarks instead of re-extracting, which prevents duplicates and gaps across deployments. A monitor task polls process states on a fixed period and records crashes and watchdog terminations. At the platform level, a container auto-updater pulls new service images in production, so the deployment tracks upstream without manual rollout. Self-deployment is what lets the other two loops run unattended: generation hands the manager a process to supervise, and healing hands it a replacement process to relaunch.

C. Self-healing

When the monitor observes a nonzero exit code it marks the wrapper `retrying` and starts a repair, unless the wrapper has exhausted its three attempts, in which case it is marked `error`. The repair call assembles the complete crashed source file, the captured stderr trace, and the last stdout lines, and asks for a complete corrected file that preserves the class structure and fixes the specific traceback. The reply passes the same static validation as initial generation, overwrites the wrapper file, increments the retry counter, and the process is restarted. Two properties distinguish this loop from generation-time self-repair [16], [17]: the trigger is a production crash that reflects real source behavior, and the unit of repair is the whole wrapper file, which lets the model restructure extraction logic rather than patch a single line.

V. Production Deployment and Metrics

We analyze a production deployment over a 149-day window ending in a full snapshot of platform state: the wrapper collection (396 documents with generated code and execution logs), per-wrapper logs, the resource and indicator stores, service logs, and container statistics. All figures below are computed by a script that accompanies the submission. The platform runs in 17 containers using about 2.0 GiB of memory at idle, or roughly 1.5 GiB excluding the optional SSO container.

A. Generation and coverage

The system generated 396 wrappers (188 XLSX, 184 API, 24 CSV) across the three domains, and they ingested 164,322 data points. Median latency from generation request to first process start was 26.8 s (90th percentile 109.8 s), including queueing, sampling, the model call, and validation. Figure 4 shows the status lifecycle with production counts.

B. Self-healing effectiveness

43 wrappers (10.9% of all generated) crashed with an unhandled exception and entered the repair loop, producing 50 repair deployments. A wrapper is healed if it never crashed again after its final repair: 42 of 43. Median crash-to-redeploy latency was 33.4 s, fully autonomous. The deployment also contains a natural baseline: before the repair loop was enabled, 36 crashes of the same exit class simply died with no repair. Table I gives the failure taxonomy and the repair-depth breakdown.

C. The autonomic boundary

The taxonomy marks where the loops stop helping. Of the 69 wrappers in a terminal `error` state, almost none reflect defective generated code: 14 never produced code (LLM quota, rate, or availability errors, or a service restart), and 54 are API pollers killed by a watchdog limit shorter than their polling interval, a configuration mismatch rather than a code defect. Exactly one error-status wrapper reflects generated code whose repair never deployed. The self-* loops absorb source heterogeneity and code defects; quota exhaustion, credential problems, and misconfigured limits remain operational work.

TABLE I
ROOT CAUSES OF THE 69 WRAPPERS WITH FINAL STATUS `error`, AND OUTCOMES OF THE SELF-HEALING LOOP (43 CRASHED WRAPPERS, 50 AUTOMATIC REPAIRS).

(a) Root causes of `error` status ($n = 69$)		
Stage	Root cause	n
Generation	Gemini daily free-tier quota (HTTP 429)	9
Generation	Gemini per-request rate limit (HTTP 429)	1
Generation	Gemini service overloaded (HTTP 503)	1
Generation	Service restart during generation	3
Runtime	Execution watchdog (3600 s), API pollers[a]	54
Runtime	Crash; repair blocked by LLM quota (429)	1
	Total	69
(b) Self-healing outcomes (43 wrappers, all crashes exit 1)		
Healed after 1 repair (crash never recurred)		37
Healed after 2 repairs		5
Still crashing after 3 repairs (retries exhausted)		1
Final status: completed / stopped / executing		28 / 8 / 3
Final status: error (later watchdog timeout)[b]		4
Wrapper-level healing success		42/43
Median crash-to-redeploy repair latency		33.4 s

[a]No traceback in stderr (53/54 empty, 1 non-fatal warning); processes were killed while idle-waiting between polling cycles because the 1 h watchdog is shorter than daily/monthly polling intervals. 48 of the 54 had already delivered data (52,372 points). [b]All four were repaired successfully and crashed no further; the subsequent `error` status stems from the watchdog limit, not from the repaired code.

VI. DISCUSSION

Three design choices carry the result: template-constrained generation bounds what the model must produce, tool-assisted probing grounds extraction in observed source behavior, and repair feedback is a real production trace. Source- agnosticism is then an emergent property of the three loops rather than a separate mechanism: the platform absorbs a new source because it can write, deploy, and repair that source's wrapper without a human writing code.

Several limitations qualify the measurements. They are observational, from one deployment, one model family, and one region; there is no contemporaneous control arm, and the pre-loop period differs in source mix and load. We cannot compare against hand-written wrappers on the same sources, which were never built. Recovery counts wrappers restored to a working state, not the semantic correctness of every extracted value; validation covers schema, not ground truth.

VII. CONCLUSION

A smart city observatory becomes source-agnostic when its wrappers manage themselves. The Observatory shows that LLM-generated wrappers can carry the ingestion layer of a production indicator platform when each wrapper is self-generating, self-deploying, and self-healing, and when the three loops share one artifact and run unattended. In 149 days the system sustained 396 wrappers and 164,322 points; of 43 wrappers that crashed, 42 were left crash-free by at most three repairs, at a median of 33.4 s each and with no operator involvement. Closing the remaining gap calls for semantic validation of extracted values and for treating environmental failures as first-class, distinct from code defects.

REFERENCES

[1] G. Wiederhold, "Mediators in the architecture of future information systems," *Computer*, vol. 25, no. 3, pp. 38–49, 1992.

[2] N. Kushmerick, "Wrapper induction: Efficiency and expressiveness," *Artificial Intelligence*, vol. 118, no. 1–2, pp. 15–68, 2000.

[3] K. Lerman, S. N. Minton, and C. A. Knoblock, "Wrapper maintenance: A machine learning approach," *Journal of Artificial Intelligence Research*, vol. 18, pp. 149–181, 2003.

[4] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[5] U. Gretzel, M. Sigala, Z. Xiang, and C. Koo, "Smart tourism: foundations and developments," *Electronic Markets*, vol. 25, no. 3, pp. 179–188, 2015.

[6] D. Buhalis and A. Amaranggana, "Smart tourism destinations," in *Information and Communication Technologies in Tourism 2014*, Z. Xiang and I. Tussyadiah, Eds. Cham: Springer International Publishing, 2013, pp. 553–564.

[7] European Commission, Directorate-General for Internal Market, Industry, Entrepreneurship and SMEs, "The European Tourism Indicator System: ETIS toolkit for sustainable destination management," Publications Office of the European Union, Luxembourg, Tech. Rep., 2016.

[8] P. Modica, A. Capocchi, I. Foroni, and M. Zenga, "An assessment of the implementation of the European Tourism Indicator System for sustainable destinations in Italy," *Sustainability*, vol. 10, no. 9, p. 3160, 2018.

[9] M. Fuchs, W. Höpken, and M. Lexhagen, "Big data analytics for knowledge generation in tourism destinations – A case from Sweden," *Journal of Destination Marketing & Management*, vol. 3, no. 4, pp. 198–209, 2014.

[10] J. A. Ivars-Baidal, M. A. Celdrán-Bernabeu, F. Femenia-Serra, J. F. Perles-Ribes, and D. Giner-Sánchez, "Measuring the progress of smart destinations: The use of indicators as a management tool," *Journal of Destination Marketing & Management*, vol. 19, p. 100531, 2021.

[11] J. A. Ivars-Baidal, J. F. Vera-Rebollo, J. Perles-Ribes, F. Femenia-Serra, and M. A. Celdrán-Bernabeu, "Sustainable tourism indicators: what's new within the smart city/destination approach?" *Journal of Sustainable Tourism*, vol. 31, no. 7, pp. 1556–1582, 2023.

[12] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[13] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1482–1494. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00129

[14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023.

[Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[16] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=KuPixIqPiq

[17] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=y0GJXRungR

[18] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language models enable simple systems for generating structured views of heterogeneous data lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023.

[19] W. Huang, Z. Gu, C. Peng, J. Liang, Z. Li, Y. Xiao, L. Wen, and Z. Chen, "AutoScraper: A progressive understanding web agent for web scraper generation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 2371–2389. [Online]. Available: https://aclanthology.org/2024.emnlp-main.141/

[20] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[21] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: Instruction-tuning local large language models for data preprocessing," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8754–8782.

[22] V. Crescenzi, G. Mecca, and P. Merialdo, "RoadRunner: Towards automatic data extraction from large web sites," in *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2001, pp. 109–118.

[23] A. H. F. Laender, B. A. Ribeiro-Neto, A. S. da Silva, and J. S. Teixeira, "A brief survey of web data extraction tools," *SIGMOD Record*, vol. 31, no. 2, pp. 84–93, 2002.

[24] E. Ferrara, P. De Meo, G. Fiumara, and R. Baumgartner, "Web data extraction, applications and techniques: A survey," *Knowledge-Based Systems*, vol. 70, pp. 301–323, 2014.

[25] N. Kushmerick, "Wrapper verification," *World Wide Web*, vol. 3, no. 2, pp. 79–94, 2000.

[26] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual modeling for ETL processes," in *Proceedings of the 5th ACM International Workshop on Data Warehousing and OLAP (DOLAP '02)*. McLean, VA, USA: ACM, 2002, pp. 14–21.

[27] P. Vassiliadis, "A survey of Extract–Transform–Load technology," *International Journal of Data Warehousing and Mining*, vol. 5, no. 3, pp. 1–27, 2009.

[28] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.

[29] M. Stonebraker, D. Bruckner, I. F. Ilyas, G. Beskales, M. Cherniack, S. B. Zdonik, A. Pagan, and S. Xu, "Data curation at scale: The Data Tamer system," in *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org, 2013. [Online]. Available: http://cidrdb.org/cidr2013/Papers/CIDR13_Paper28.pdf

[30] T. Furche, G. Gottlob, L. Libkin, G. Orsi, and N. W. Paton, "Data wrangling for big data: Challenges and opportunities," in *Proceedings of the 19th International Conference on Extending Database Technology (EDBT 2016)*. Bordeaux, France: OpenProceedings.org, 2016, pp. 473–478.